

Group 12 - Project plan

Thien Tu Ngo i6373732
Kristian Bek i6388692
Malo Coquin i6302990
Ilie Lîsenco i6394209
Mohamed Abdelhalim Saber Eldeeb i6373947
Aditya Singha i6393599

October 2025

Contents

1	Project description	2
2	All tasks breakdown	2
2.1	Unity environment	2
2.2	ML-Agents integration	2
2.3	Data collection	3
2.4	Machine learning implementation	3
3	Primary research questions	3
4	Data collection plan	4
4.1	Target Variables	4
4.2	Feature Variables	4
4.3	DRL algorithm to be used	4
4.4	Number of training runs	5
4.5	Realistic data collection schedule	5
5	Detailed schedule of activities	5
5.1	Phase 1	6
5.2	Phase 2	6
5.3	Phase 3	7
6	High level project architecture overview	7
7	Risk assessment, mitigation and contingency plan	8
7.1	Technical risks and mitigation	8
7.2	Data Collection risks and mitigation	8
7.3	Contingency plan for scenarios	9

8	Defining success criteria	9
9	Task distribution	10

1 Project description

We want to develop an autonomous racing system, consisting of a simple, highly abstracted car agent which learns to complete laps around custom designed tracks using Deep Reinforcement Learning. Our primary focus will be to collect data from training runs and apply supervised machine learning techniques to predict training outcomes.

1. **Agent:** Car agent.
2. **Goal:** Complete a lap around a track without human input.
3. **Environment:** Track simulator.
4. **Type of environment:** Deterministic and episodic.

2 All tasks breakdown

2.1 Unity environment

1. Create a simple car agent with basic physics. Implement high levels of abstraction.
2. Design three track configurations (Easy, Medium and Hard).
3. Implement dense reward function with checkpoint based reward
4. Develop observation space inside environment - we can use ray case sensors + velocity + angular velocity
5. Implement continuous action space. (Actions consist of steering and throttle).

2.2 ML-Agents integration

1. We have to setup training configs using different hyperparameter combinations.
2. We have to set episode (lap) reset logic and success criteria.
3. We may integrate Tensorboard logging for real-time monitoring of the training runs.

2.3 Data collection

1. We write Python scripts to automate experiment management system.
2. Episode-level and aggregated training metrics are to be logged in CSV format.
3. We structure data schema with validation scripts.
4. We have to implement a system to detect failed runs and auto-retry.
5. Integration with BCS2720-project-data-collection-2025-2026.

2.4 Machine learning implementation

1. Predictive modelling using scikit-learn
2. Turning training run log data into properly formatted data to feed into models.
3. Model interpretation using SHAP values and feature importance in case of black-box models such as XGBoost.

3 Primary research questions

1. Can we predict the wall-clock training time required for a racing agent to achieve first lap completion based on hyperparameters and hardware specifications?
2. How does track complexity affect the minimum number of training steps required to reach consistent performance?

4 Data collection plan

4.1 Target Variables

Variable	Type	Description	Collection Method
time_to_first_lap	Continuous	Wall-clock seconds until first successful lap	Monitor episode completion flag
steps_to_first_lap	Continuous	Training steps until first successful lap	Episode counter at first success
final_avg_lap_time	Continuous	Average lap time over last 100 episodes	Rolling average calculation
success_rate_final	Continuous	% successful laps in last 100 episodes	Binary success tracking
training_stability	Continuous	Std. dev. of episode rewards (last 500 steps)	Statistical aggregation
peak_ram_usage_mb	Continuous	Maximum RAM consumption during training	Python <code>psutil</code> monitoring

4.2 Feature Variables

Category	Variable	Values/Range
Hardware	cpu_cores	Integer (measured)
	ram_gb	Integer (measured)
	gpu_name	String (e.g., "RTX3060", "None")
	os_type	Windows, Linux, MacOS
Hyperparameters	learning_rate	0.0001, 0.0003, 0.001
	batch_size	128, 512, 2048
	buffer_size	10240, 51200
	network_width	64, 128, 256
	network_depth	2, 3
	max_steps	500000, 1000000
Environment	track_difficulty	easy, medium, hard
	num_checkpoints	Integer (measured per track)
Training Config	random_seed	Integer (for reproducibility)
	curriculum_enabled	Boolean

4.3 DRL algorithm to be used

We will be using solely Proximal Policy Gradient (PPO) algorithm for the following reasons - runs reasonably fast by leveraging vector (parallel) environments and naturally works well with different action spaces, therefore supporting a variety of games. It also has good sample efficiency compared to algorithms such as DQN.

4.4 Number of training runs

In our project, curriculum refers to training the same car agent on progressively harder versions of a track, allowing it to first master easier scenarios before tackling full-difficulty environments. We will only consider `curriculum_enabled` as a variable in our training runs if we are able to implement the curriculum well enough (which would change stability and speed of training: if ON, agents often reach success faster and more reliably) or we have time for more training runs. Therefore, we provide two calculations:

Case 1: Without `curriculum_enabled`

$$\begin{aligned}\text{Unique configurations} &= 3 \times 3 \times 2 \times 6 \times 2 \times 3 \\ &= 648 \text{ unique configurations}\end{aligned}$$

With 3 random seeds per configuration:

$$648 \times 3 = 1944 \text{ total training runs}$$

Case 2: With `curriculum_enabled`

$$\begin{aligned}\text{Unique configurations} &= 3 \times 3 \times 2 \times 6 \times 2 \times 3 \times 2 \text{ (curriculum on/off)} \\ &= 1296 \text{ unique configurations}\end{aligned}$$

With 3 random seeds per configuration:

$$1296 \times 3 = 3888 \text{ total training runs}$$

It is not feasible to achieve 3888 training runs in our schedule, hence the variable is of low importance at the moment.

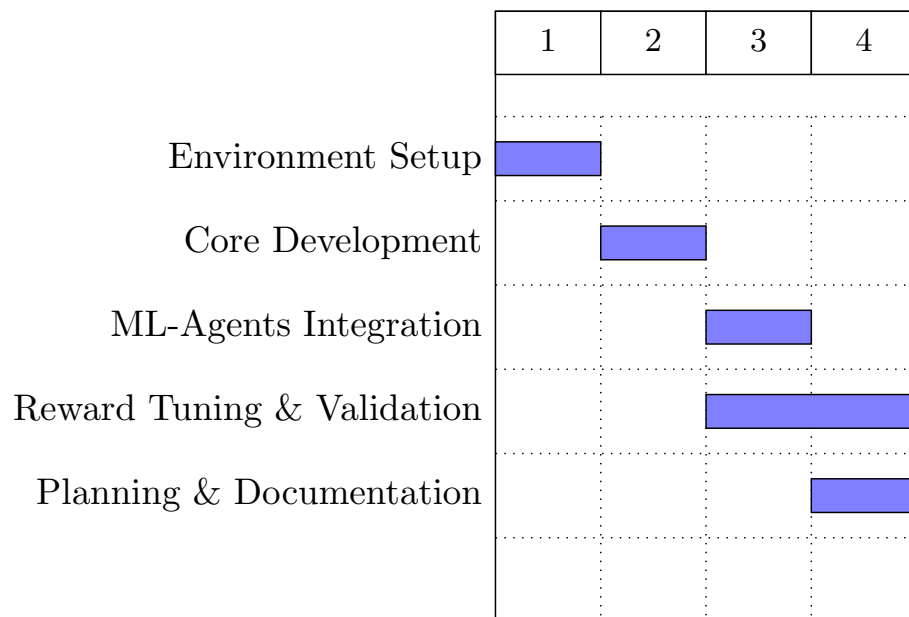
4.5 Realistic data collection schedule

1. Phase 2: Target 400 runs (100/week with 6 machines running parallel)
2. Phase 3 (Weeks 9-13): Target 600 runs (120/week)
3. Total achievable: 1,000 runs minimum
4. Ideally: 1,500+ runs if training efficiency improves

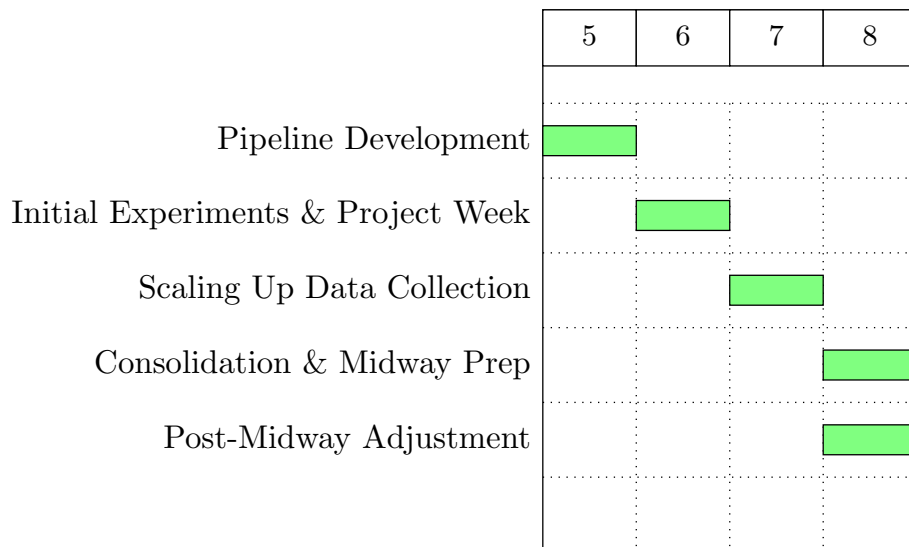
5 Detailed schedule of activities

The following gantt charts show the high-level tasks to be completed, per phase, in each week. (read 1, 2, 3, etc., as weeks)

5.1 Phase 1



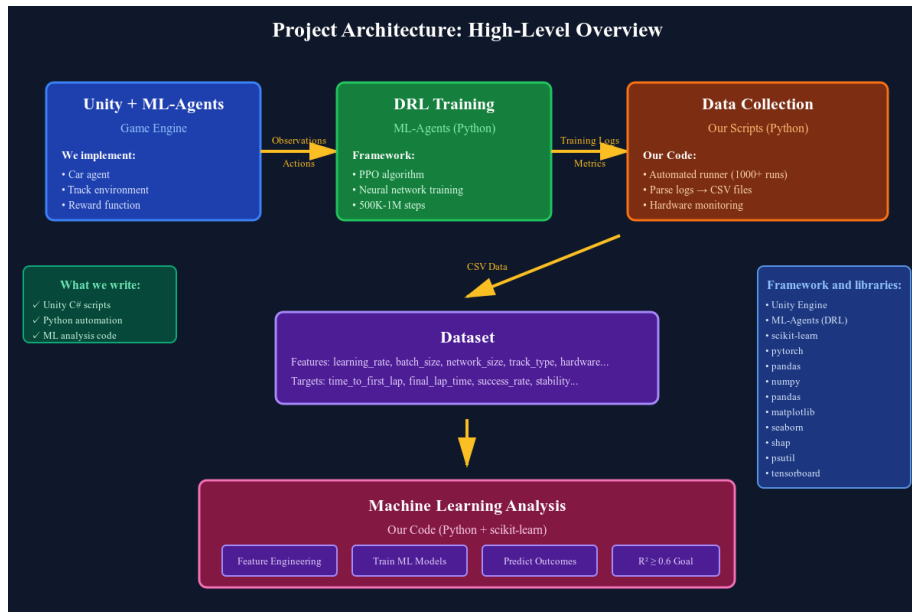
5.2 Phase 2



5.3 Phase 3

	9	10	11	12	13
Data Prep & Baseline Models					
Advanced Model Development					
Data Collection & Model Refinement					
Visualization & Reporting					
Finalization & Presentation					

6 High level project architecture overview



7 Risk assessment, mitigation and contingency plan

7.1 Technical risks and mitigation

- ML-Agents installation failures (High impact): We'll use recommended fork; create detailed installation script; venv setup; maintain troubleshooting document.
- Car never learns to complete a lap (CRITICAL): Week 3.5 dedicated to reward tuning; implement curriculum learning; use dense checkpoint-based rewards; consult with instructor if stuck.
- Training runs take more than 12 hours each (High): We'll use simpler tracks initially; reduce max_steps to 500K; optimize Unity physics timestep; run overnight on all 6 machines.
- Unstable training convergence (High): Multiple random seeds (3 per config); early stopping after 100K steps if no progress; save checkpoints every 50K steps.
- Data corruption/loss (High): Automated backups every 2 days; git LFS for large files; separate data repository; validation scripts run after each batch.
- TensorFlow/PyTorch version conflicts (Medium): Pin exact package versions in requirements.txt; use venv exclusively; document known compatibility issues.

7.2 Data Collection risks and mitigation

- Only able to collect 500 runs instead of 1000+ (Medium): We start data collection early; prioritize diverse hyperparameters over repeats; reduce number of unique configs of experiments if needed.
- Reward function produces poor training signal (CRITICAL): Iterative testing Week 3-3.5; small-scale validation (50K steps) before full runs; maintain reward design notebook.
- Inconsistent data schema across machines (High): Centralized data validation script; automated schema checker in experiment runner; single "data logger" class used by all.
- 30%+ experiment failures due to crashes (Medium): Automatic retry with same config; flag chronic failures; investigate common failure patterns; reduce max_steps if stability improves.
- Hardware performance too variable (Low): We'll record exact hardware specs with each run; use hardware as features; analyze hardware impact separately.

7.3 Contingency plan for scenarios

- **If car doesn't learn by end of Week 3.5 (>1 week behind schedule):**
 - We'll switch to simpler task (reach goal, not complete lap)
 - We'll use pre-built ML-Agents example (3DBall) and modify it
 - We'll consult instructor immediately
- **If data collection is >2 weeks behind schedule:**
 - We'll reduce number of unique configurations
 - We'll accept 2 repeats instead of 3
 - We'll use shared BCS2720 dataset more heavily
 - We'll extend max_steps reduction for faster runs
- **If ML models perform poorly ($R^2 < 0.3$):**
 - We'll simplify prediction targets (binary classification: success/failure instead of continuous time)
 - We'll focus on feature engineering (polynomial features, interactions)
 - We'll collect more targeted data (fill gaps in hyperparameter space)
 - We'll use ensemble methods (voting regressors)
- **If team member drops or falls significantly behind:**
 - Immediate task reallocation in next meeting
 - Pair programming sessions to catch up
 - We'll reduce individual scope, maintain team deliverables
 - We'll escalate to instructor if needed

8 Defining success criteria

Minimum Viable Product:

- Car autonomously completes laps on at least ONE track type
- ≥ 400 valid training runs collected and logged
- Predictive model achieves $R^2 \geq 0.5$ on test set for at least one target variable
- All code documented with docstrings and README
- Experiments reproducible via configuration files (no manual code changes needed)

Ideally:

- Car completes laps on all three track types with $\leq 10\%$ failure rate
- $\geq 1,000$ training runs collected
- $R^2 \geq 0.7$ for primary prediction target
- Successfully integrate data from other BCS2720 groups
- Deploy model as web API for real-time predictions

9 Task distribution

Everyone in the group will be responsible for contributing data to the dataset during all phases. Group members with more capable GPUs will be tasked with performing training runs of tracks of harder difficulties, more training steps, etc.

- Thien Tu Ngo: Reward function, architecture and data schema design (phase 1), writing scripts to train agent in environment (phase 2), feature engineering and hyperparameter tuning (phase 3)
- Kristian Bek: Unity agent and environment setup (phase 1), automating data flow from Unity to dataset (phase 2), code documentation and final report compilation (phase 3)
- Malo Coquin: Unity agent and environment setup (phase 1), general data pipeline support (phase 2), code documentation and final report compilation (phase 3)
- Ilie Lisenco: Unity agent and environment setup (phase 1), general data pipeline support (phase 2), feature engineering and hyperparameter tuning (phase 3)
- Mohamed Abdelhalim Saber Eldeeb: Reward function, architecture and data schema design (phase 1), Unity game troubleshooting and support (phase 2), data analysis and hypothesis testing (phase 3)
- Aditya Singha: Reward function, architecture and data schema design (phase 1), preliminary data analysis (phase 2), data analysis and hypothesis testing (phase 3)